Contents

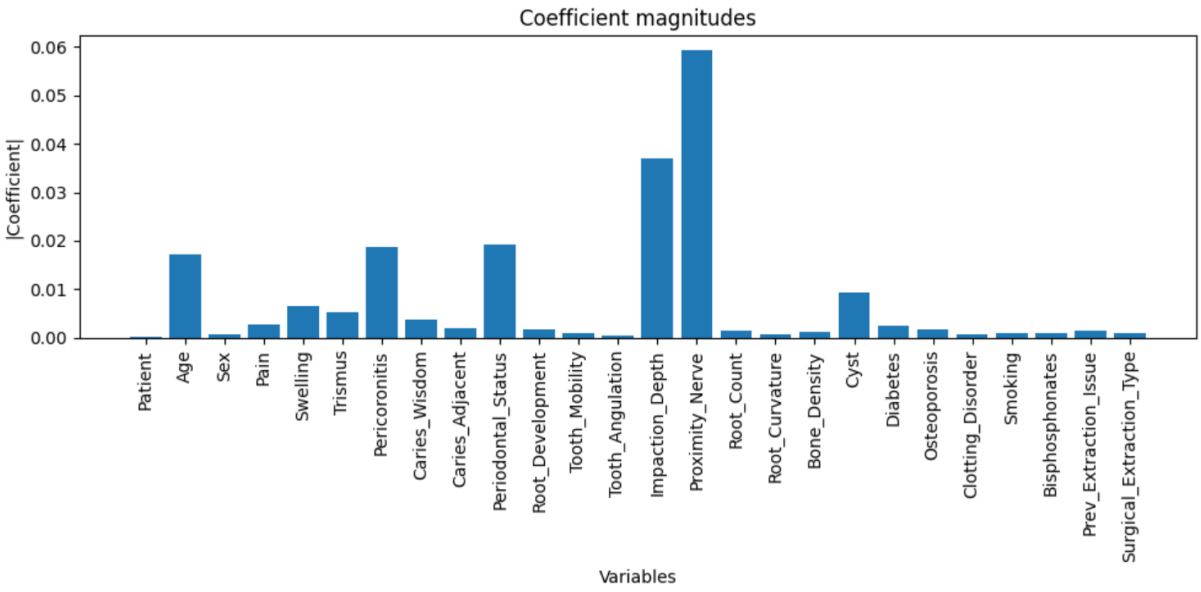
1 Groundwork

At this time in the project we took a step back and investigated some simpler models alongside the Multi-Layer-Perceptron (MLP) approach.
Although the MLP model achieved strong predictive performance, simpler models are often preferred when they provide sufficient accuracy, since they are easier to interpret and explain. In particular, linear regression models allow direct inspection of the learned coefficients, making it possible to identify which variables are associated with increased or decreased complication risk. In our "groundwork"we use the dataset generated by Jonas/Smoothy, who worked on our project previously. The dataset contains 50000 data points where 3000 data points is used as test set.
The work can be found in the "Groundwork"folder.

1.1 Multipel Regression

1.1.1 Binary Target

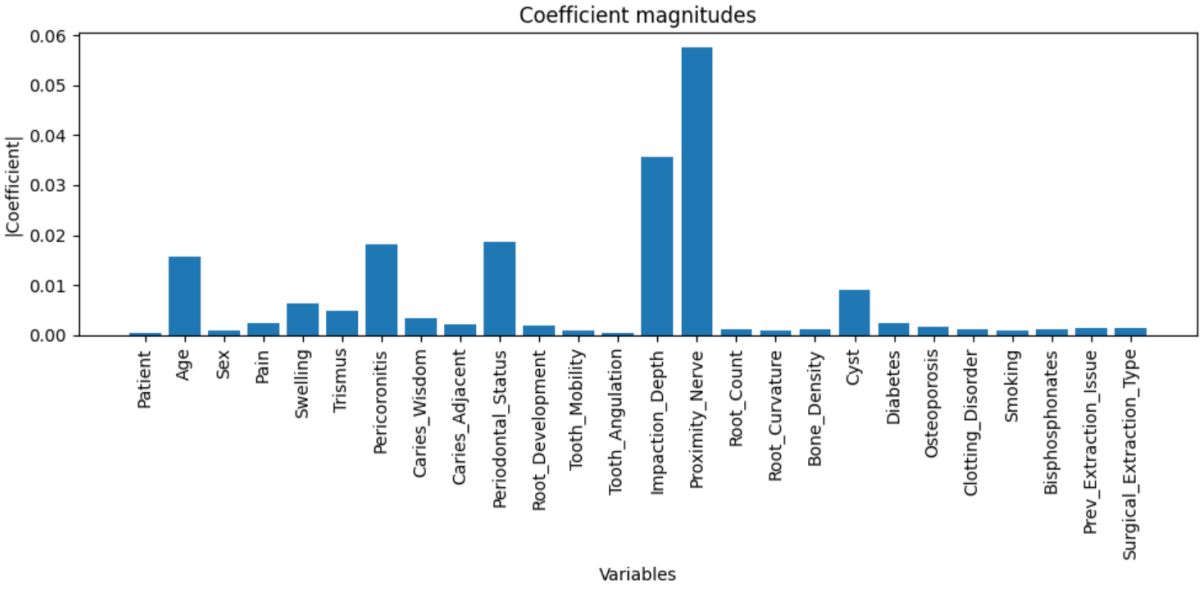
We begin by fitting a multiple linear regression model to investigate the relationship between the input features and a new binary target variable where a value of 1 indicates that a patient experienced one or more complications, while 0 indicates no complications.
Before training the model, the input features were standardized to ensure that the regression coefficients are directly comparable across variables.
The figure below shows the regression coefficients for the model.



The regression coefficients indicate that variables such as proximity to the nerve and impaction depth have the strongest influence on the predicted complication risk.

We now extend the analysis to a federated setting. Instead of training a single regression model on the full dataset, a separate multiple linear regression model is trained locally on each client again using the binary target variable.

After local training, the regression coefficients are aggregated using a weighted average weighted by the number of samples at each client. This produces a global federated model.
The figure below shows the regression coefficients for the model.



We see that the coefficients of the federated multibel regression are close to those of the centralized setup.

1.1.2 Four models

We now extend the analysis by training one regression model for each of the four complication types.
The overall method remains the same in both the centralized and federated settings.
Since the models now produce separate predictions for each complication, the predicted risk scores can be categorized into the classes low, medium, and high using percentile threshold. This makes it possible to evaluate the models using the F1-macro score across the three risk categories.

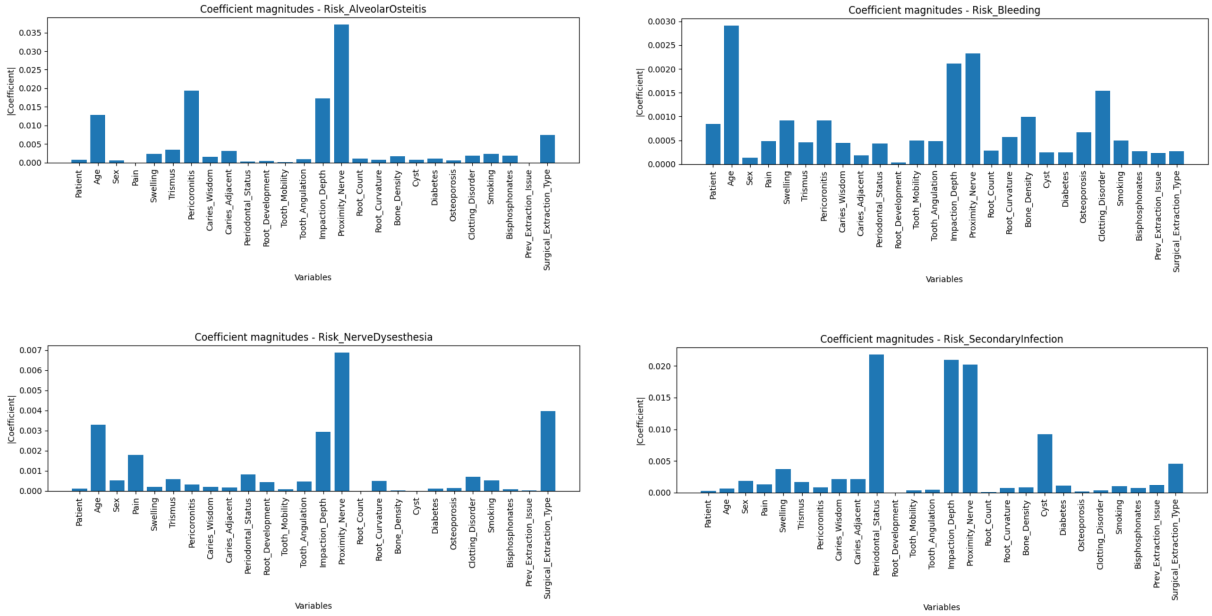
```
--- Risk_AlveolarOsteitis ---
True column: Risk_Category_AlveolarOsteitis
Pred column: Risk_AlveolarOsteitis_group
Class 0: Precision=0.765, Recall=0.793, F1=0.779
Class 1: Precision=0.545, Recall=0.531, F1=0.538
Class 2: Precision=0.715, Recall=0.709, F1=0.712
Macro F1 = 0.676

--- Risk_SecondaryInfection ---
True column: Risk_Category_SecondaryInfection
Pred column: Risk_SecondaryInfection_group
Class 0: Precision=0.702, Recall=0.732, F1=0.717
Class 1: Precision=0.550, Recall=0.529, F1=0.539
Class 2: Precision=0.747, Recall=0.746, F1=0.746
Macro F1 = 0.667

--- Risk_NerveDysesthesia ---
True column: Risk_Category_NerveDysesthesia
Pred column: Risk_NerveDysesthesia_group
Class 0: Precision=0.745, Recall=0.813, F1=0.778
Class 1: Precision=0.607, Recall=0.553, F1=0.579
Class 2: Precision=0.746, Recall=0.757, F1=0.751
Macro F1 = 0.703

--- Risk_Bleeding ---
True column: Risk_Category_Bleeding
Pred column: Risk_Bleeding_group
Class 0: Precision=0.549, Recall=0.640, F1=0.591
Class 1: Precision=0.376, Recall=0.403, F1=0.389
Class 2: Precision=0.642, Recall=0.531, F1=0.582
Macro F1 = 0.520

==============================
CENTRALIZED FINAL F1 = 0.642
==============================
```

(a) Centralized

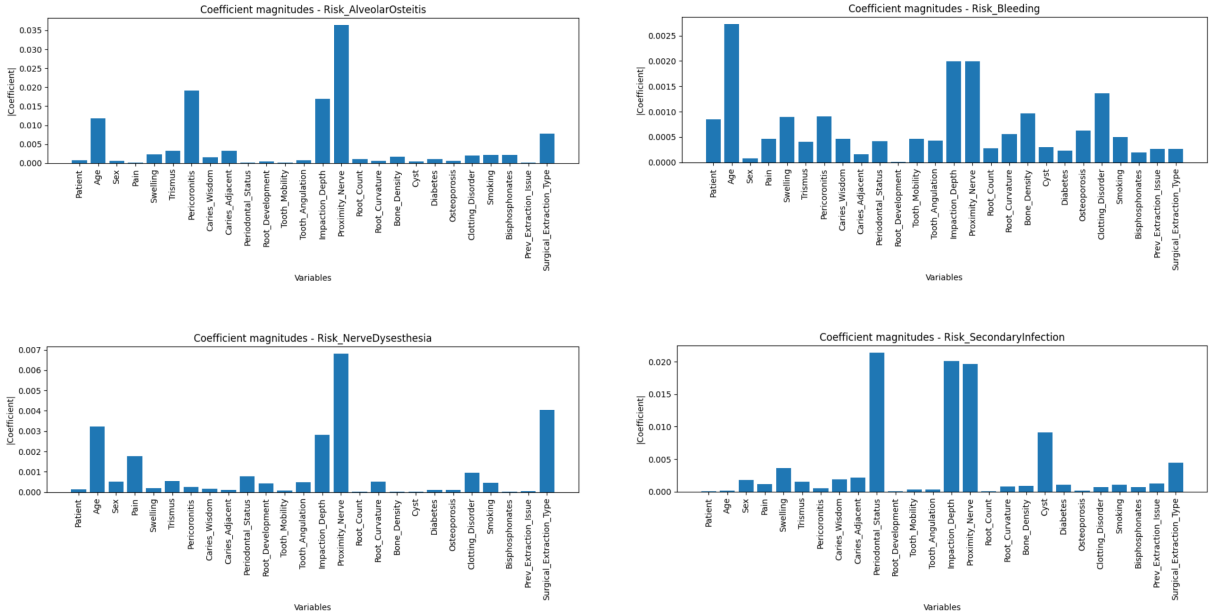
```
--- Risk_AlveolarOsteitis ---
True column: Risk_Category_AlveolarOsteitis
Pred column: Risk_AlveolarOsteitis_group
Class 0: Precision=0.765, Recall=0.793, F1=0.779
Class 1: Precision=0.522, Recall=0.508, F1=0.515
Class 2: Precision=0.698, Recall=0.692, F1=0.695
Macro F1 = 0.663

--- Risk_SecondaryInfection ---
True column: Risk_Category_SecondaryInfection
Pred column: Risk_SecondaryInfection_group
Class 0: Precision=0.686, Recall=0.715, F1=0.700
Class 1: Precision=0.538, Recall=0.518, F1=0.528
Class 2: Precision=0.733, Recall=0.732, F1=0.732
Macro F1 = 0.653

--- Risk_NerveDysesthesia ---
True column: Risk_Category_NerveDysesthesia
Pred column: Risk_NerveDysesthesia_group
Class 0: Precision=0.742, Recall=0.810, F1=0.775
Class 1: Precision=0.621, Recall=0.566, F1=0.592
Class 2: Precision=0.763, Recall=0.774, F1=0.768
Macro F1 = 0.712

--- Risk_Bleeding ---
True column: Risk_Category_Bleeding
Pred column: Risk_Bleeding_group
Class 0: Precision=0.501, Recall=0.584, F1=0.539
Class 1: Precision=0.369, Recall=0.395, F1=0.382
Class 2: Precision=0.616, Recall=0.510, F1=0.558
Macro F1 = 0.493

==============================
FINAL F1 (mean over risks) = 0.630
==============================
```

(b) Federated

Overall, the results are promising. The federated approach achieves performance close to the centralized setup. In addition, the achieved macro F1-score is relatively high compared to the performance observed for the base MLP model (0.748 for centralized and 0.6414 for federated). The plots below show the regression coefficients for the four centralized regression models, one for each complication type.



The coefficient plots indicate that the different complication types are explained by different input features. While some variables, such as impaction depth, appear important across multiple complications, others have large coefficients only for specific risks, suggesting that each complication is influenced by distinct underlying factors. Below we have a similar plot for the federated models:



We see that for the federated setup, the coefficient magnitude are almost identical to the centralized model.

1.2 Logistic Regression

After the promising results obtained with multiple regression, we turned our attention to logistic regression. Since the target variable is binary, logistic regression seemed a natural choice.

1.2.1 Centralized

We begin with a centralized setup where all training data are pooled together and used to train a single logistic regression model. As in the multiple regression experiments, all input variables are standardized before training. After fitting the model, the predicted probabilities can be interpreted as estimated complication risks. the predicted probabilities are divided into low-, medium-, and high-risk groups using tertile thresholds, and model performance is evaluated using the macro F1-score. We get the following F1 scores:

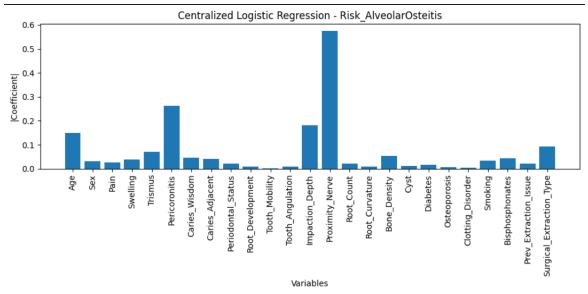
```
Risk_AlveolarOsteitis: 0.670
Risk_SecondaryInfection: 0.682
Risk_NerveDysesthesia: 0.727
Risk_Bleeding: 0.528
```

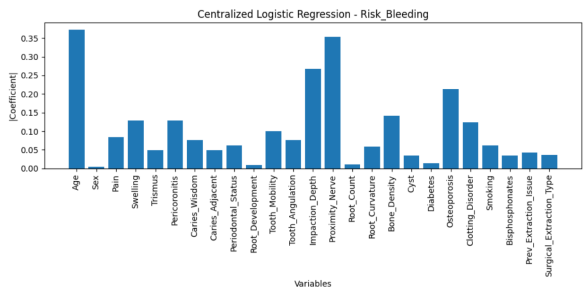
```
============================
CENTRALIZED LOGISTIC FINAL F1 = 0.652
============================
```

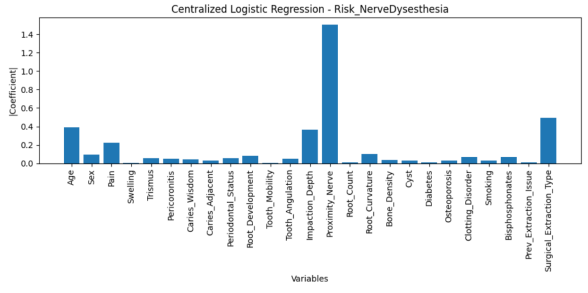
The results obtained with logistic regression are slightly better than those observed for multiple regression. The overall macro F1-score increases from 0.642 to 0.652, corresponding to an improvement of approximately 0.01.

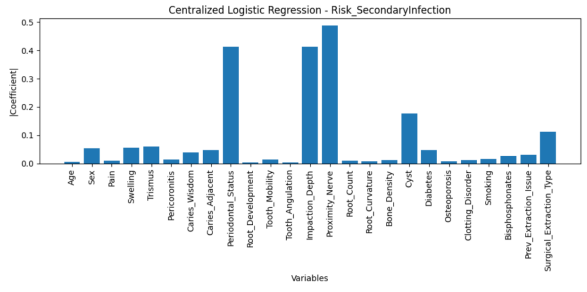
The largest improvement is observed for Nerve Dysesthesia, where the macro F1-score increases from 0.703 to 0.727. Secondary Infection also improves from 0.667 to 0.682, while Bleeding increases slightly from 0.520 to 0.528. For Alveolar Osteitis, the performance decreases from 0.676 to 0.670.

Overall, the differences are relatively small. This suggests that the two models produce similar rankings of patients, this i properly due to the final evaluation is based on assigning patients to low-, medium-, and high-risk groups using percentile thresholds. This means, that the ordering of predictions is more important than their exact numerical values, leading to similar performance. As for multiple regression, we also visualize the magnitude of the learned coefficients:









1.2.2 Federated

We now consider the federated case. Instead of having all observations in a single dataset, the data is distributed across clients and training is performed locally and then aggregated as a weighted average. We get the following F1 Scores:

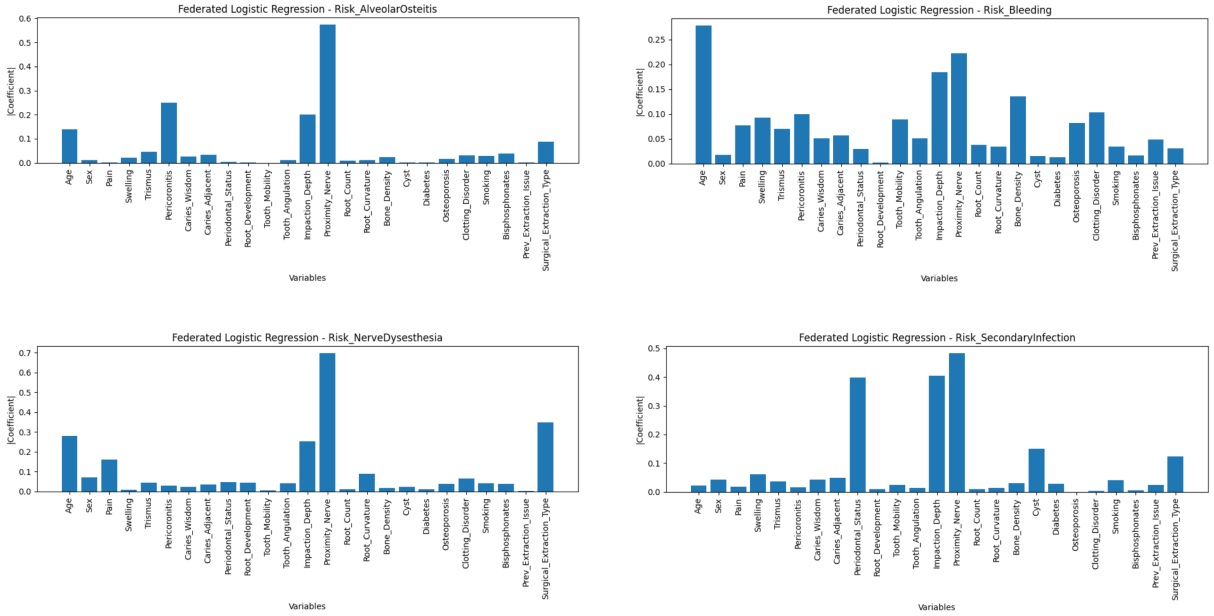
```
Risk_AlveolarOsteitis: 0.673
Risk_SecondaryInfection: 0.682
Risk_NerveDysesthesia: 0.716
Risk_Bleeding: 0.516
```

```
==============================
FEDERATED LOGISTIC FINAL F1 = 0.647
==============================
```

We observe a small improvement compared to federated multiple regression. The overall macro F1-score increases from 0.638 to 0.647.

For the individual complications, Alveolar Osteitis improves from 0.663 to 0.673 (+0.010), Secondary Infection improves from 0.653 to 0.682 (+0.029), Nerve Dysesthesia improves from 0.712 to 0.716 (+0.004), and Bleeding improves from 0.493 to 0.516 (+0.023).

Overall, the differences are relatively small. The differences are relatively small. This is not surprising, since our evaluation mainly depends on how well the models rank patients by risk. Both multiple regression and logistic regression appear to produce very similar rankings, leading to nearly identical macro F1-scores. we also visualize the magnitude of the learned coefficients:



The coefficient plots reveal several interesting patterns. First, the same variables consistently receive large coefficients across both multiple regression and logistic regression. In particular, variables related to nerve proximity, impaction depth, and root morphology appear among the most influential predictors. This suggests that the two modeling approaches identify largely the same underlying risk factors.

1.3 Feature Selection

The results obtained with multiple regression and logistic regression are very similar, with logistic regression providing only a small improvement in macro F1-score.

For the feature selection experiments, we therefore continue with multiple regression. Multiple regression is computationally simpler and can be fitted directly, whereas logistic regression relies on an iterative optimization procedure. To keep the analysis computationally manageable we only do the following steps for the complication Alveolar_osteitis, doing it for the other complications would only require running the code again for these.
Due to the computational cost of several of the feature selection techniques, we in some cases reduce the size of the dataset in order to make the experiments manageable to run.

1.3.1 Forward and Backward selection

We perform both forward and backward feature selection using a significance level of 0.05 on the model without interaction-terms. This results in selecting the following features:

const	0.088872
Age	0.012257
Trismus	0.004064
Pericoronitis	0.020224
Caries_Adjacent	-0.002893
Impaction_Depth	0.016990
Proximity_Nerve	0.037978
Surgical_Extraction_Type	0.007296

(a) Backward

const	0.088872
Proximity_Nerve	0.037978
Pericoronitis	0.020224
Impaction_Depth	0.016990
Age	0.012257
Surgical_Extraction_Type	0.007296
Trismus	0.004064
Caries_Adjacent	-0.002893

(b) Forward

Both techniques select the same variables. The selected variables are largely consistent with what was observed in the coefficient-magnitude plots for Alveolar Osteitis, where the same features appeared to have the strongest influence on the prediction.

We are not only interested in which features on their own are important, but also which interactions between features. Therefore we want to do feature selection on all interaction pairs. Since this is computationally heavy we limit the dataset to only 2500 data points. We select the following features using backward and forward selection:

```
['Age',
 'Age_x_Proximity_Nerve',
 'Bone_Density_x_Cyst',
 'Bone_Density_x_Prev_Extraction_Issue',
 'Caries_Adjacent_x_Prev_Extraction_Issue'
 'Cyst_x_Clotting_Disorder',
 'Diabetes_x_Surgical_Extraction_Type',
 'Impaction_Depth',
 'Impaction_Depth_x_Proximity_Nerve',
 'Pericoronitis',
 'Pericoronitis_x_Bone_Density',
 'Pericoronitis_x_Proximity_Nerve',
 'Periodontal_Status_x_Osteoporosis',
 'Proximity_Nerve',
 'Proximity_Nerve_x_Bone_Density',
 'Root_Count',
 'Root_Count_x_Cyst',
 'Root_Development',
 'Root_Development_x_Proximity_Nerve',
 'Sex_x_Cyst',
 'Sex_x_Root_Count',
 'Swelling_x_Prev_Extraction_Issue',
 'Swelling_x_Root_Count',
 'Swelling_x_Surgical_Extraction_Type',
 'Tooth_Angulation_x_Clotting_Disorder',
 'Trismus',
 'Trismus_x_Cyst',
 'Trismus_x_Proximity_Nerve']
```

(a) Backward

```
['Age',
 'Age_x_Proximity_Nerve',
 'Bone_Density_x_Cyst',
 'Bone_Density_x_Prev_Extraction_Issue',
 'Caries_Adjacent_x_Prev_Extraction_Issue',
 'Caries_Wisdom_x_Osteoporosis',
 'Cyst_x_Clotting_Disorder',
 'Diabetes_x_Surgical_Extraction_Type',
 'Impaction_Depth',
 'Impaction_Depth_x_Proximity_Nerve',
 'Pericoronitis',
 'Pericoronitis_x_Bone_Density',
 'Pericoronitis_x_Proximity_Nerve',
 'Proximity_Nerve',
 'Proximity_Nerve_x_Bone_Density',
 'Root_Count',
 'Root_Count_x_Cyst',
 'Root_Development',
 'Root_Development_x_Proximity_Nerve',
 'Sex_x_Cyst',
 'Sex_x_Root_Count',
 'Swelling_x_Prev_Extraction_Issue',
 'Swelling_x_Root_Count',
 'Swelling_x_Surgical_Extraction_Type',
 'Tooth_Angulation_x_Clotting_Disorder',
 'Trismus',
 'Trismus_x_Cyst',
 'Trismus_x_Proximity_Nerve']
```

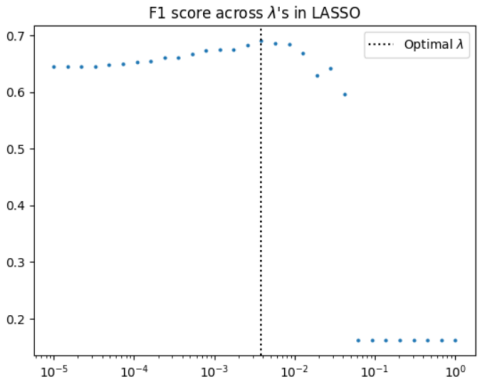
(b) Forward

Both methods selects 28 features with only a few differences. The selected features suggest that interaction effects play an important role in predicting Alveolar Osteitis. Several interaction terms are not eliminated, indicating that the effect of one variable often depends on the value of

another variable.
However, forward and backward selection only work for the centralized setup and not for federated, since that would require aggregating p-values (which does not make sense). Therefore we also look into using LASSO-regression, since LASSO-regression can be adapted to the federated setup.

1.3.2 LASSO

We now investigate LASSO regression as a method for feature selection in both the federated and centralized setup. By adding regularization during training, LASSO can reduce the number of selected features. Note that we are now working on the full dataset.
First we do it on a centralized model, in order to have something to compare the federated setup results to. In order to do this we first need to choose a good λ-value, an we therefore do a sweep off 30 different λ - values. The plot below shows the F1-macro score vs lambda value:

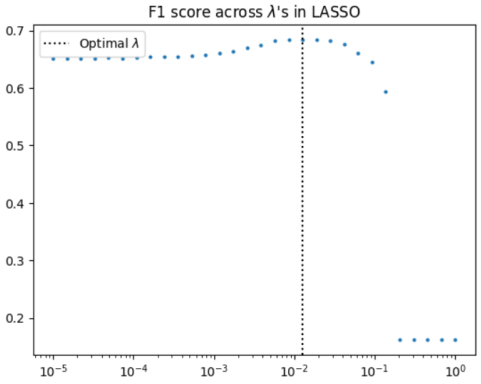


We now investigate the performance of the best observed λ-value. However, as seen in the previous plot, a fairly large range of λ-values gives similar performance, indicating that the model is relatively robust to the exact choice of regularization strength. The lambda value with the best F1-score is 0.00385 which has the following performance and selected features:

```
{'per_class': {0: {'TP': np.int64(794),
   'FP': np.int64(212),
   'FN': np.int64(171),
   'precision': np.float64(0.7892644135188867),
   'recall': np.float64(0.8227979274611399),
   'f1': np.float64(0.8056823947234907)},
  1: {'TP': np.int64(565),
   'FP': np.int64(432),
   'FN': np.int64(462),
   'precision': np.float64(0.5667001003009027),
   'recall': np.float64(0.5501460564751705),
   'f1': np.float64(0.558300395256917)},
  2: {'TP': np.int64(710),
   'FP': np.int64(287),
   'FN': np.int64(298),
   'precision': np.float64(0.7121364092276831),
   'recall': np.float64(0.7043650793650794),
   'f1': np.float64(0.7082294264339153)}},
 'macro_f1': np.float64(0.6907374054714411)}
```

```
['Age_x_Impaction_Depth',
 'Age_x_Proximity_Nerve',
 'Age_x_Surgical_Extraction_Type',
 'Trismus_x_Proximity_Nerve',
 'Pericoronitis_x_Impaction_Depth',
 'Pericoronitis_x_Proximity_Nerve',
 'Impaction_Depth_x_Proximity_Nerve',
 'Impaction_Depth_x_Surgical_Extraction_Type']
```

The centralized LASSO regression with lambda value 0.00385 selects 8 features and achieves a performance of 0.69, which is an improvement from 0.676, which we achieved for Alveolar osteitis without interactions. In comparison the centralized MLP-model achieves a performance of 0.818 for alveolar osteitis.
We then do it for the federated model. The plot below shows the sweep of λ-values:



We now investigate the performance of the best observed λ-value. Again we observe, that a fairly large range of λ-values gives similar performance, indicating that the model is relatively robust to the exact choice of regularization strength.The lambda value with the best F1-score is 0.012689 which has the following performance and selected features:

```
['Age',
 'Impaction_Depth',
 'Proximity_Nerve',
 'Age_x_Pericoronitis',
 'Age_x_Impaction_Depth',
 'Age_x_Proximity_Nerve',
 'Age_x_Surgical_Extraction_Type',
 'Trismus_x_Impaction_Depth',
 'Trismus_x_Proximity_Nerve',
 'Trismus_x_Root_Curvature',
 'Pericoronitis_x_Tooth_Mobility',
 'Pericoronitis_x_Impaction_Depth',
 'Pericoronitis_x_Proximity_Nerve',
 'Pericoronitis_x_Surgical_Extraction_Type',
 'Caries_Wisdom_x_Diabetes',
 'Periodontal_Status_x_Proximity_Nerve',
 'Root_Development_x_Proximity_Nerve',
 'Impaction_Depth_x_Proximity_Nerve',
 'Impaction_Depth_x_Surgical_Extraction_Type',
 'Proximity_Nerve_x_Root_Curvature',
 'Proximity_Nerve_x_Surgical_Extraction_Type']
```

```
{'per_class': {0: {'TP': np.int64(788),
   'FP': np.int64(212),
   'FN': np.int64(177),
   'precision': np.float64(0.788),
   'recall': np.float64(0.816580310880829),
   'f1': np.float64(0.8020356234096693)},
  1: {'TP': np.int64(553),
   'FP': np.int64(447),
   'FN': np.int64(474),
   'precision': np.float64(0.553),
   'recall': np.float64(0.5384615384615384),
   'f1': np.float64(0.5456339417858905)},
  2: {'TP': np.int64(707),
   'FP': np.int64(293),
   'FN': np.int64(301),
   'precision': np.float64(0.707),
   'recall': np.float64(0.7013888888888888),
   'f1': np.float64(0.704183266932271)}},
 'macro_f1': np.float64(0.6839509440426103)}
```

The federated lasso regression with lambda value 0.012689 selects 21 features (so a lot more than in the centralized model) and achieves a performance of 0.68, which is an improvement from 0.66, which we achieved in the multiple regression model for Alveolar osteitis without interactions. The federated MLP-model achieves a F1-macro score of 0.7152 for alveolar osteitis.
It should be noted that the λ values are selected directly based on performance on the test set. From a methods perspective, this is not fully correct, since the test set should ideally only be used for final evaluation and not for hyperparameter tuning. As a result, the reported performances should be interpreted as an upper bound on the achievable performance.

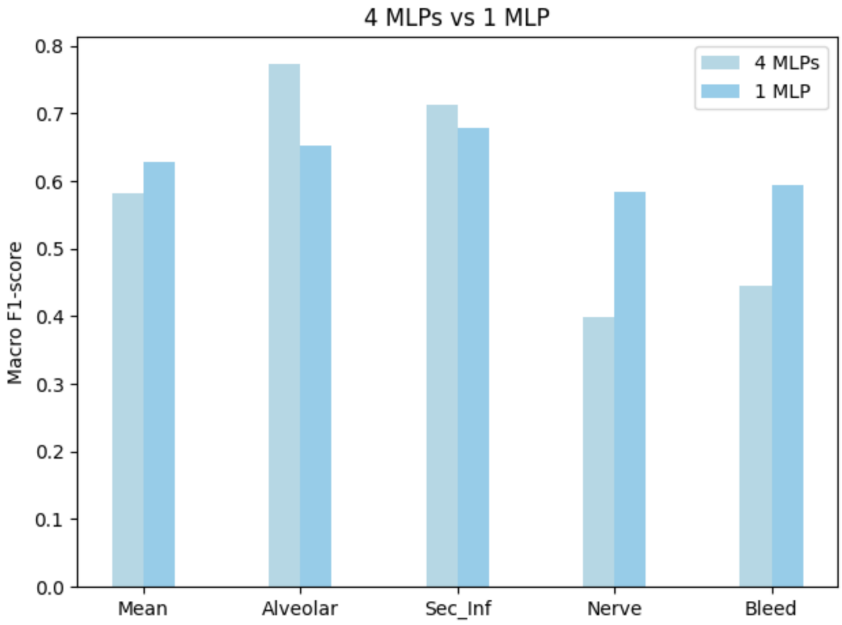
2 Optimizing MLP-architecture

In this section we experiment with different architectures of the MLP. The motivation for this is that we through our simple models have seen that different features in the dataset have different importance for different complications. Because of this a single MLP with only fully connected layers might not be optimal. We try to train four independent MLP's and we also try to train an MLP, that has some shared layers and then brancing out into individual layers for each complication. We also try different arhcitectures of the single MLP
The work can be found in src/fdrp/analysis/results/MLP_arkitektur_results.
In the following section (regarding MLP architecture) we only did it in the federated setup, since

this is the model we are interested in optimizing, and training more in centralized as well will take a long time.

2.1 Four independent MLP's

We train four different MLP's - one for each complication - and compute the F1-score for each comlication as well as the mean F1-score. The four MLP's have same architecture as the setup with one MLP.

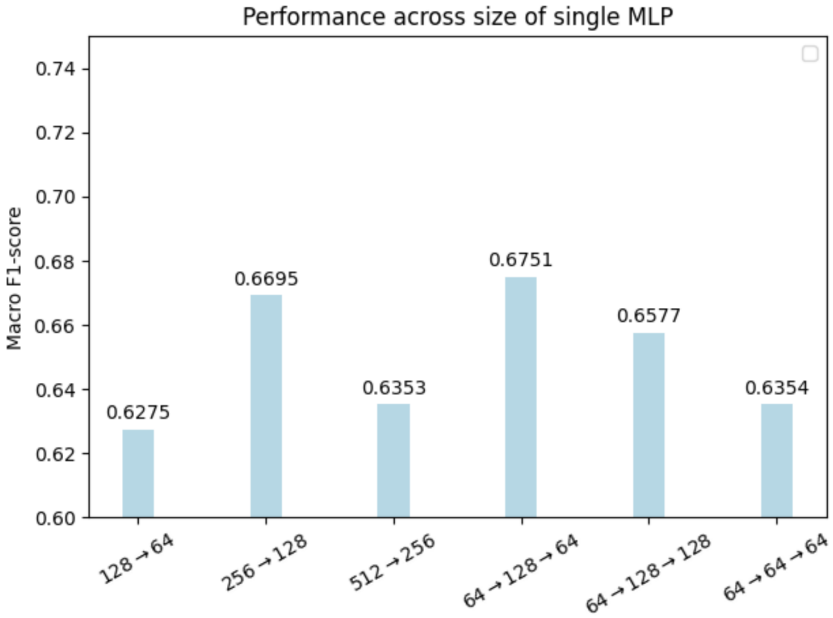


We observe above that there is greater variance in performance across complications for the four MLPs. Performance is very poor for the rare categories, "Nerve" and "Bleeding". We hypothesize that this is because the rare complications contain too few positive cases for the model to learn the patterns properly. Therefore, it is advantageous to use a shared model that can learn general features that are relevant across complications, such as nerve_proximity.
From this, we conclude that four individual MLPs perform poorly and are not a suitable architecture.

2.2 1 model, no branching, different sizes of layers

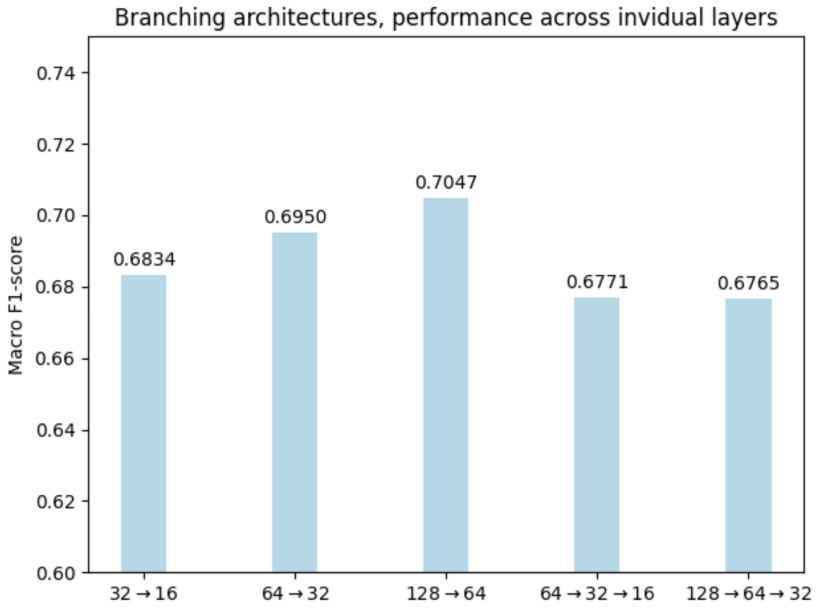
To rule out if an increaed performance in the branching models is just due to a sheer increase in size of the model, we did some experiments on the single MLP and its size and architecture. We tried with different number of hidden layers and different number of neurons in these hidden layers. Below are the results. The notation used is for example $128 \rightarrow 64$ which means a hidden layer of size 128 into a hidden layer of size 64 - giving us a total of 2 hidden layers.

We see that the best performance is achieved with an architecture of $64 \rightarrow 128 \rightarrow 64$. We also see that the performance drops when the size of the model is increased to the architecture of $64 \rightarrow 128 \rightarrow 128$, and thus we do not experiment further with bigger layers in the three-layer architecture. The same goes for the 2-layer architecture where we see a drop, when the model gets too big.



2.3 Models with branching

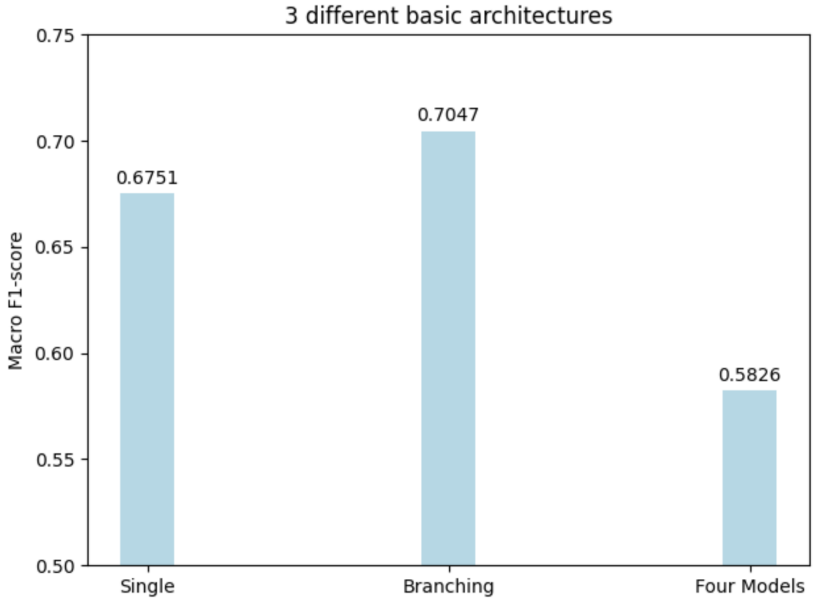
We now try to implement models that have a common backbone but later branch out in layers that are not fully connected. The motivation for this is, that the model can use the training data points from all the complications to tune the backbone - and the finetune the branched parts so they learn the specific features that are relevant for each complication. This is an attempt to get the best of both worlds. In the experiments we have done, all the architectures have a common backbone of $128 \rightarrow 64$. They then branch out into individual layers that have the size denoted in the graph. Below we see the results:



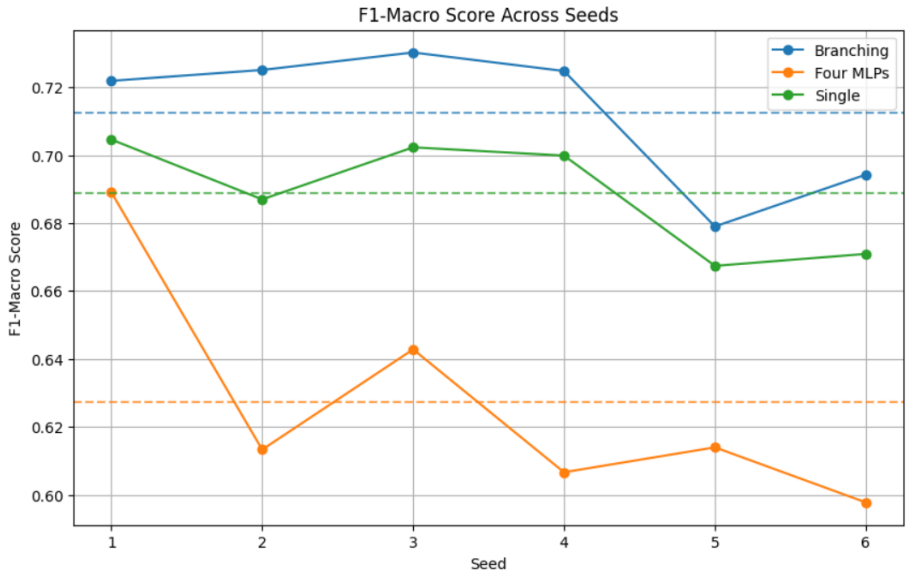
Here we see a really good performance, in fact the best we have seen across all federated models in the federated setup. That is the case for the architecture with branched layer of $128 \rightarrow 64$. Thus we conclude that the branching strategy is actually working.

2.4 Comparison of 4 MLP's, no branching single MLP and branching MLP

To sum up we here show the best performance for seen for each of the architectures:

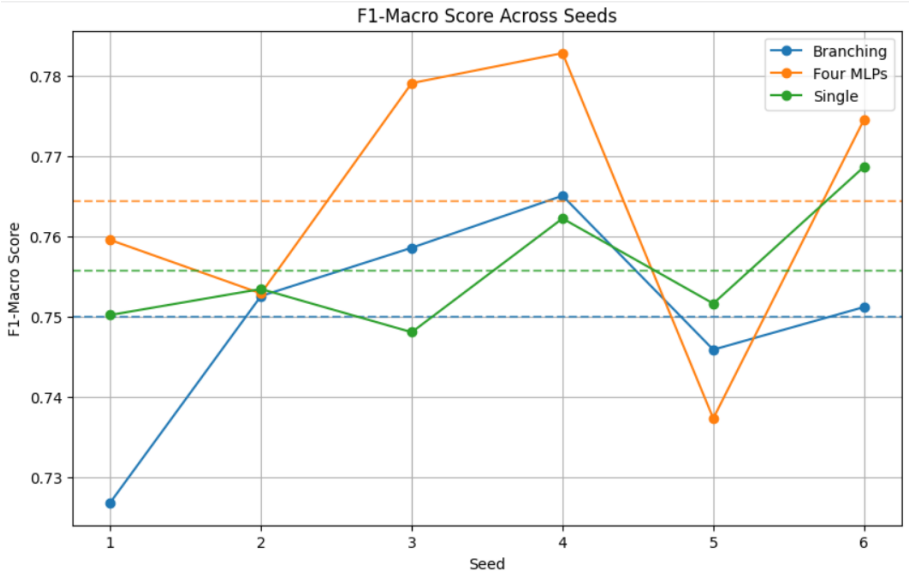


Here we see that the branching architecture is a bit better than the single MLP, and that these two are clearly better than the 4 MLP's. We hypothesize that is the due to two reasons: 1) There are different features that are important for explaining the different complications. 2) Some of the complications are very rare, so an MLP trained only on this complication does not have enough positive cases to learn the patterns that lead to these. Thus the combination a common backbone and an individual branching are solving both of these issues best at the same time. To investigate whether the improved performance of the branching architecture was caused by a favorable random seed (originally we used seed 42), we trained the best-performing architectures across multiple seeds. We used $\beta_L = 1$ and $\beta_Q = 1$ for moderate label and quantity skew. The figure below shows the F1-macro score for each seed together with the average performance across seeds.



(The work for the plot above can be found in src/fdrp/analysis/branching tests) We observe that the branching architecture consistently achieves a higher F1-macro score than the single

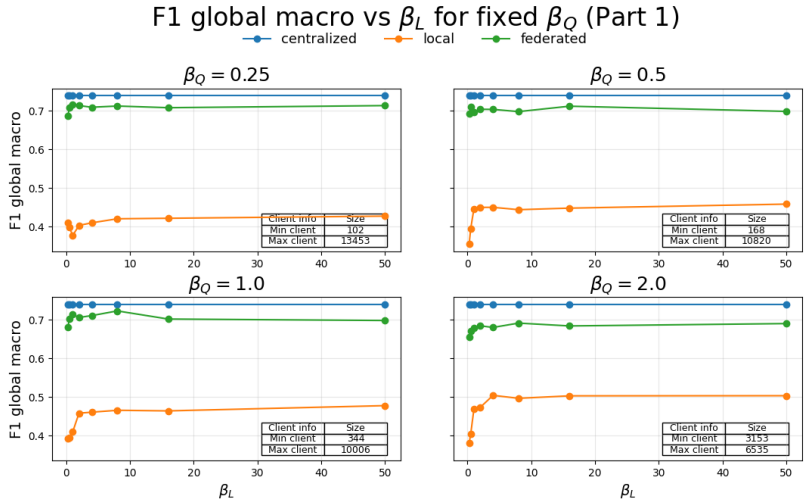
MLP across all tested seeds. The mean performance difference between the branching and single MLP architectures is approximately 0.023, while the standard deviation of the differences across seeds is only approximately 0.009. This suggests that the improvement obtained from branching is not caused by a favorable data splits. We also note that the four independent MLPs perform worse. For comparison we do the same for centralized:

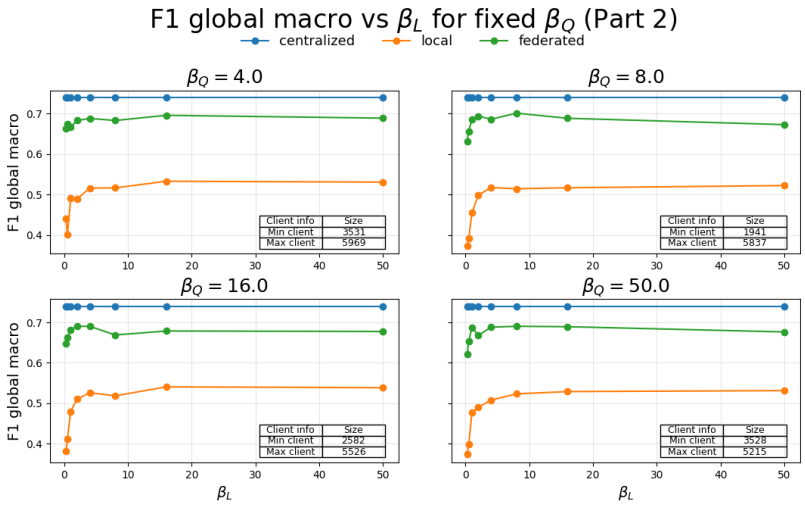


For the centralized setting, the results are less clear. Unlike in the federated setup, the branching architecture does not consistently outperform the single MLP across all seeds. In fact, the performances of the branching and single MLP architectures are very similar on average, and for some seeds the single MLP slightly outperforms the branching architecture.

Beta sweep of the best model

To conclude this part we make a beta sweep using the model with the best federated performance (the one with branching). You can read more about beta sweep in the beta sweep section:

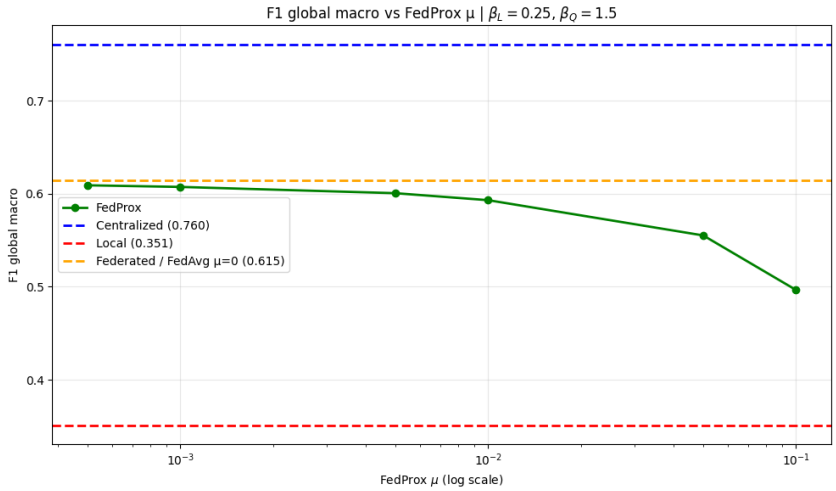


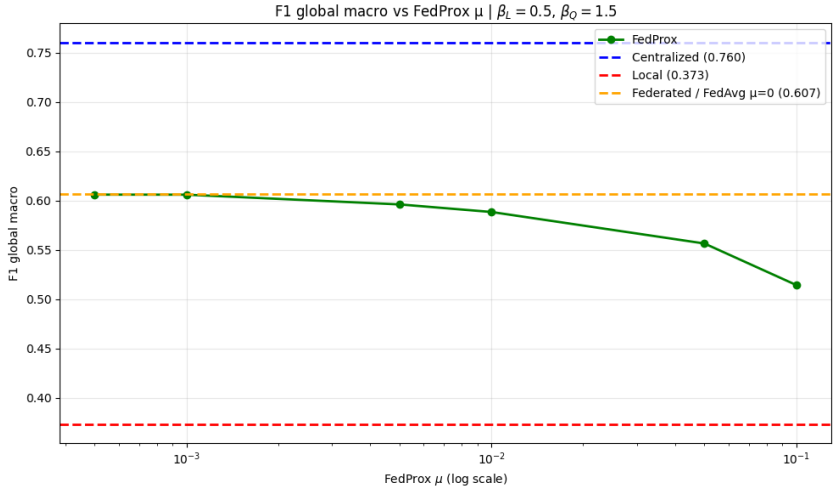


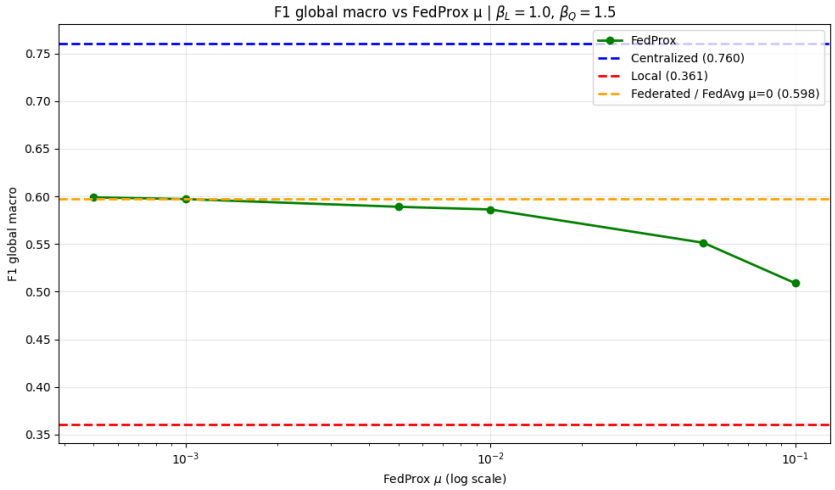
We see, that in general the results from the federated model is very close to that of the centralized model, across many different data configurations.

3 FedProx

We also experimented with implementing another aggregation method, FedProx. In the following we use the base MLP ($128 \to 64$) since FedProx was implementet before we testet additional architectures.
FedProx is implemented in src/fdrp/ml/federated/train. The code for the plots is in src/fdrp/analysis/plot_fedprox and src/fdrp/analysis/FedProx_mu_search.
Below is three plots of the FedProx model. The x-axis has the μ-value. For reference we have centralized, local and FedAvg show by dashed lines. In the three plots we have used a moderate quantity skew of $\beta_Q = 1.5$. In the three plot we vary the label skew with the values 0.25,0.5 and 1.0:







These findings indicate that the additional proximal constraint introduced by FedProx does not yield any improvement in scenarios where client heterogeneity is relatively moderate.

4 Beta Sweep Results

In this section, we investigate the performance of our MLP model under different levels of data heterogeneity in a federated learning setting. So far, we have worked on a synthetic dataset generated by Jonas/Smoothy, who previously worked on the project.

Our data partitioning framework distributes the data across clients using two parameters: β_L and β_Q. The parameter β_Q controls the quantity skew, i.e., how many data points each client receives. The parameter β_L controls the label skew by influencing the distribution of patients with complications across clients.

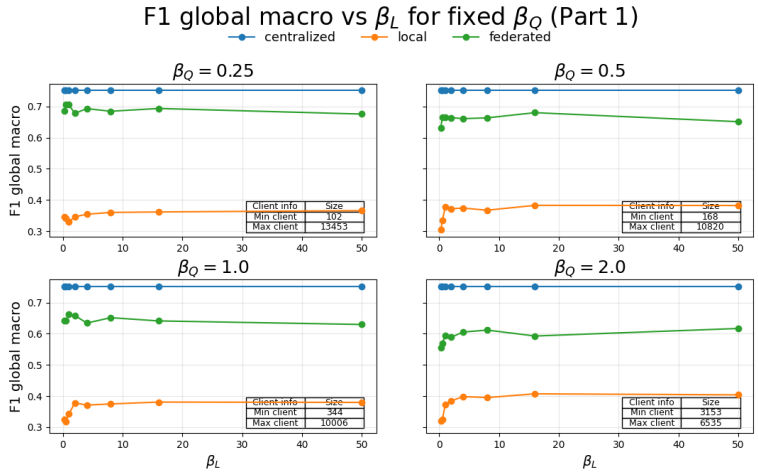
Smaller values of β_L and β_Q result in a more heterogeneous federated setup, while larger values lead to more homogeneous client datasets. This enables us to systematically analyze how varying levels of heterogeneity affect the performance of our federated MLP model.
The work can be found in src-analysis.

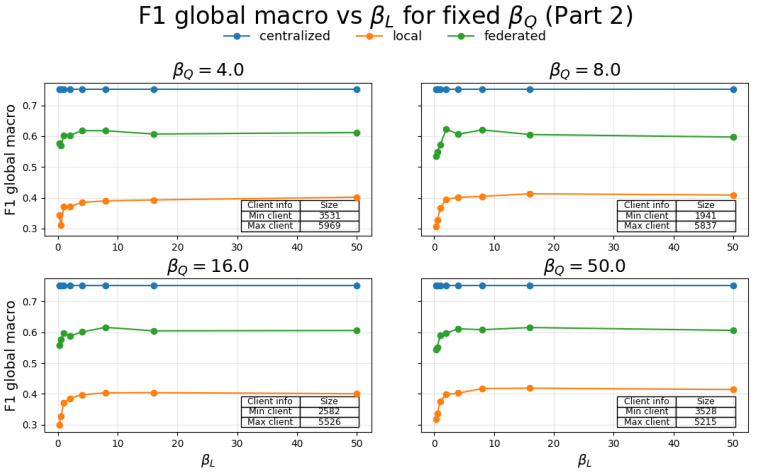
4.0.1 Label Skew

We run a grid search/beta sweep. Using seed 42 (and a dataset of size 50000 with 3000 test data points), with the values:

$$\beta_Q = [0.25, 0.5, 1.0, 2.0, 4.0, 8.0, 16.0, 50.0]$$
$$\beta_L = [0.25, 0.5, 1.0, 2.0, 4.0, 8.0, 16.0, 50.0]$$

The eight plots below show the F1 macro metric (computed using global thresholds) as a function of label skew (β_L) for a fixed quantity skew (β_Q).



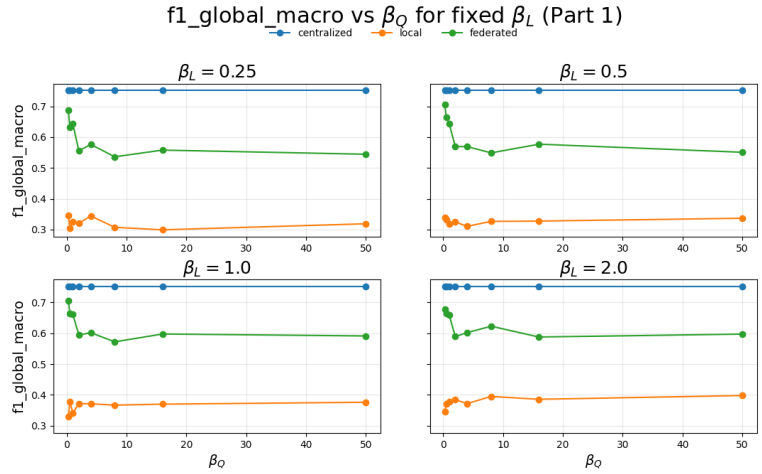


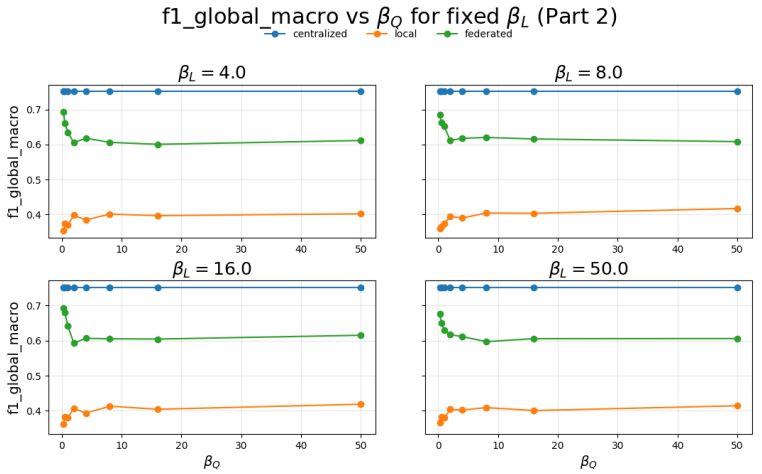
The results show a clear performance ranking between the three training paradigms. Centralized training consistently achieves the highest F1 score, followed by federated learning, while locally trained models perform substantially worse. This confirms that federated learning successfully leverages information across clients, significantly outperforming purely local training.

Varying the label skew parameter β_L generally has a limited impact on model performance for small values of β_Q, where quantity skew is high. However, as β_Q increases and the client datasets become more balanced in size, the effect of label skew becomes more important. In particular, very low values of β_L lead to a decrease in federated performance for $\beta_Q \geq 2$. This suggests that the federated model is relatively robust to moderate levels of label heterogeneity, but extreme label skew can harm performance when the quantity distribution across clients is more homogeneous. High quantity skew (small β_Q's) results in a high performance of the federated model. This is likely due to the fact that few/one client has most of the data points resulting in a model close to the centralized model.

4.0.2 Quantity Skew

The eight plots below show the F1 macro metric (computed using global thresholds) as a function of quantity skew (β_Q) for a fixed label skew (β_L).

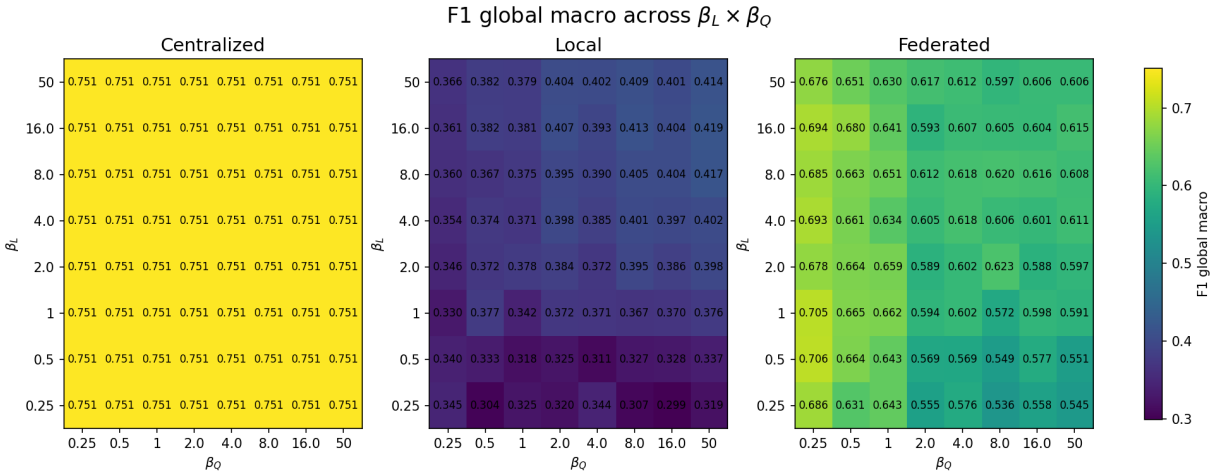




The plot shows the same trends as before, but now from the perspective of varying β_Q for fixed values of β_L.

4.0.3 Interaction Plot

The heatmaps summarize the global macro F1 performance across combinations of β_L and β_Q for the centralized, local, and federated setups.

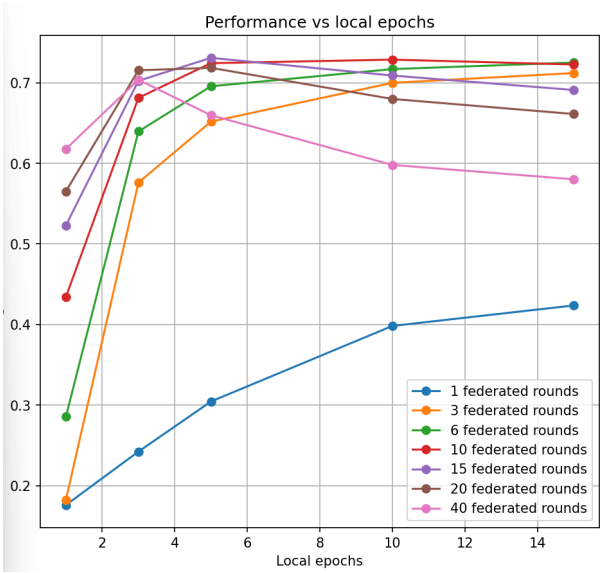


5 Choice of hyperparameters

Before beginning the main experiments, we first investigated the effect of several important hyperparameters. In particular, we tested different values for the test set size, the number of local epochs, and the number of federated rounds. The goal was to find a reasonable balance between model performance and computational cost. Since many experiments had to be repeated multiple times under different partitioning settings, it was important that the training procedure achieved good performance without long training time. These analysis was made under a new seed (seed 10).

5.1 Federated model

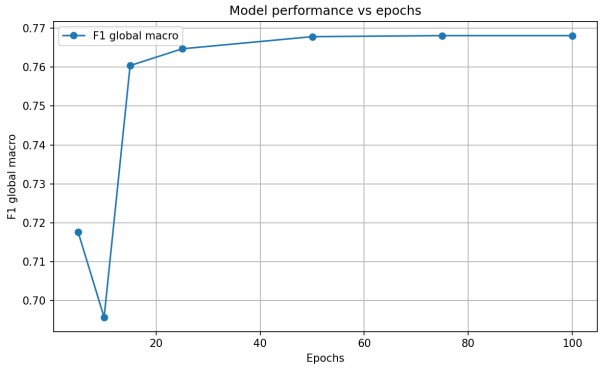
For the federated model, we first performed a small hyperparameter sweep over the number of local epochs and federated rounds. The figure below shows the performance for different combinations of local epochs and federated rounds:



We figured that the original choice of 6 federated rounds and 5 local epochs gave a good balance between performance and training time. In some cases, slightly better performance could be achieved by increasing the number of rounds or epochs, but this also made the experiments take longer to run.

5.2 Centralized

For the centralized model, we also performed a small sweep over the number of training epochs. As shown in the figure below, the performance improved quickly in the beginning, but the gains became very small after around 15 epochs. We therefore chose 15 epochs as a reasonable compromise between model performance and training time.



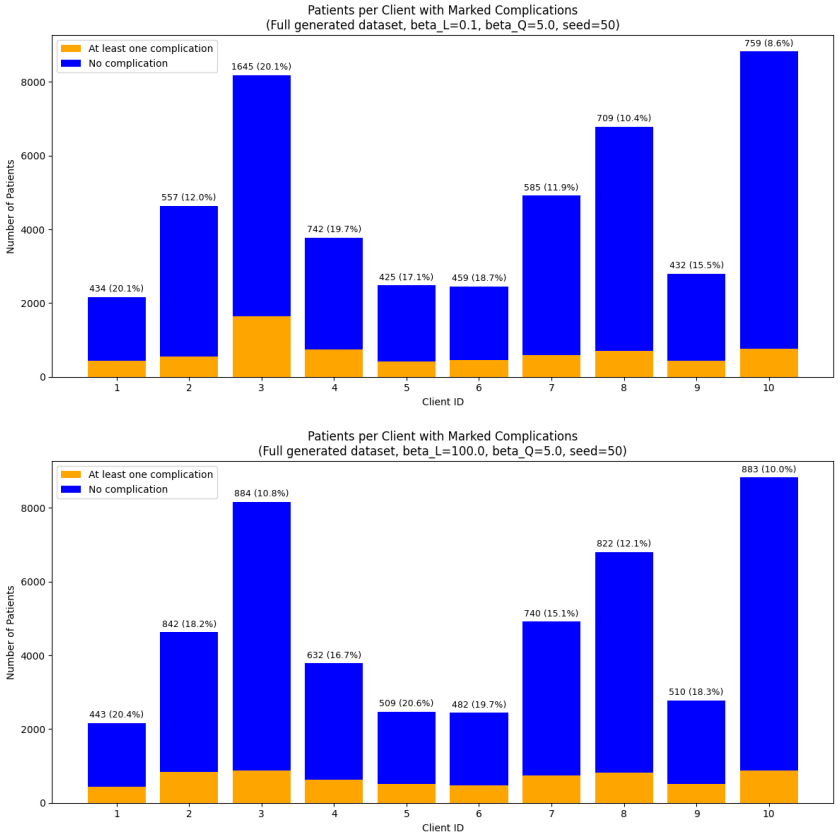
6 Data partitioning

The data partitioning work can be found in src/fdrp/data_generation/partitioning

6.1 Old Partitioning

The previous partitioning approach (Implemented by "Jonas/Smoothy"who previously worked on the project) was based on a standard Dirichlet partitioning method inspired by NIID-Bench (https://github.com/Xtra-Computing/NIID-Bench). First, the dataset was split based on the target labels, i.e. whether a patient had at least one complication or not. For each label, a Dirichlet distribution parameterized by β_L was sampled across all clients. The sampled proportions determined how large a share of samples with that label each client would receive. Low values of β_L resulted in uneven label distributions between clients, while high values produced more similar client distributions.

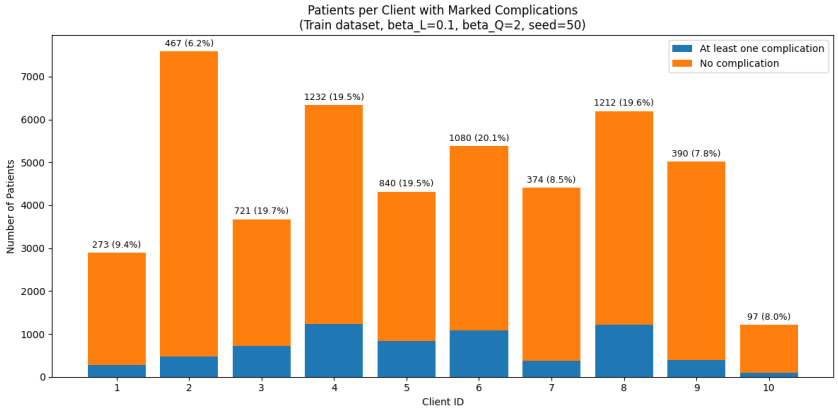
After the label partitioning step, quantity skew was introduced separately. Another Dirichlet distribution, controlled by β_Q, was sampled to determine the desired dataset size for each client. Samples were then either removed or added from each client in order to match these target sizes as closely as possible. The main issue with this approach was that label skew and quantity skew were handled in two separate steps. First, β_L was used to create different label distributions across clients. However, afterwards β_Q was applied to change the size of each client dataset. Since this step removed or retained samples after the label distribution had already been created, the final client distributions were no longer controlled only by β_L. This resultet in β_L having little to no control:

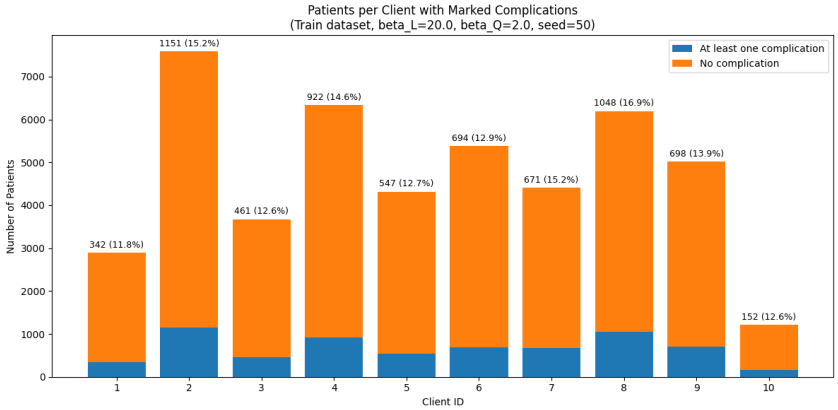


From the figures, we can see that when $\beta_L = 100$, where the data should be close to homogeneous, the percentage of patients with complications still varies from approximately 12.1% to 20.6% across clients. On the other hand, when $\beta_L = 0.1$, where the data should be highly heterogeneous, the percentages vary from approximately 10.4% to 20.1%. This suggests that the difference between homogeneous and heterogeneous settings was less extreme than expected.

6.2 New Partitioning

The final partitioning approach was redesigned together with ChatGPT to jointly handle both quantity skew and label skew. First, the desired dataset size for each client is sampled using a Dirichlet distribution controlled by β_Q. Afterwards, label distributions for each client are sampled using another Dirichlet distribution controlled by β_L.
Instead of applying these steps sequentially, the method uses Iterative Proportional Fitting (IPF) to iteratively adjust the allocation until both constraints are satisfied simultaneously:
each client receives the correct number of samples, and the global label counts are preserved exactly.
This avoids the issue from the previous approach, where quantity skew could unintentionally dominate the final label distributions.





The new partitioning approach appears to provide much better control over the final client distributions. When $\beta_L = 20$, where the data should be relatively homogeneous, the percentage of patients with complications is fairly similar across clients, ranging from approximately 11.8% to 16.9%. In contrast, when $\beta_L = 0.1$, the client distributions become much more heterogeneous, with percentages ranging from approximately 6.2% to 20.1%.